

Exercise 04

```
lan@vihanVM in ~ via C v16.1.1-gcc via v5.42.2 via v3.14.5 took 29s210ms
) gcc -fopenmp -O2 -o exe4_lab5 exe4_lab5.c && OMP_NUM_THREADS=1 ./exe4_la
b5
r = 1, k = 1, thread # = 0
r = 1, k = 2, thread # = 0
r = 1, k = 3, thread # = 0
r = 1, k = 4, thread # = 0
r = 1, k = 5, thread # = 0
r = 1, k = 6, thread # = 0
r = 1, k = 7, thread # = 0
r = 1, k = 8, thread # = 0
r = 1, k = 9, thread # = 0
r = 1, k = 10, thread # = 0
r = 2, k = 1, thread # = 0
r = 2, k = 2, thread # = 0
r = 2, k = 3, thread # = 0
r = 2, k = 4, thread # = 0
r = 2, k = 5, thread # = 0
r = 2, k = 6, thread # = 0
r = 2, k = 7, thread # = 0
r = 2, k = 8, thread # = 0
r = 2, k = 9, thread # = 0
r = 2, k = 10, thread # = 0
r = 3, k = 1, thread # = 0
r = 3, k = 2, thread # = 0
r = 3, k = 3, thread # = 0
r = 3, k = 4, thread # = 0
r = 3, k = 5, thread # = 0
r = 3, k = 6, thread # = 0
r = 3, k = 7, thread # = 0
r = 3, k = 8, thread # = 0
r = 3, k = 9, thread # = 0
r = 3, k = 10, thread # = 0
r = 4, k = 1, thread # = 0
r = 4, k = 2, thread # = 0
r = 4, k = 3, thread # = 0
r = 4, k = 4, thread # = 0
r = 4, k = 5, thread # = 0
r = 4, k = 6, thread # = 0
r = 4, k = 7, thread # = 0
r = 4, k = 8, thread # = 0
r = 4, k = 9, thread # = 0
r = 4, k = 10, thread # = 0
r = 5, k = 1, thread # = 0
r = 5, k = 2, thread # = 0
r = 5, k = 3, thread # = 0
r = 5, k = 4, thread # = 0
r = 5, k = 5, thread # = 0
```

```
r = 5, k = 5, thread # = 0
r = 5, k = 6, thread # = 0
r = 5, k = 7, thread # = 0
r = 5, k = 8, thread # = 0
r = 5, k = 9, thread # = 0
r = 5, k = 10, thread # = 0
Time taken: 0.000045 seconds
```

```
vihan@vihanzVM in ~ via C v16.1.1-gcc via v5.42.2 via v3.14.5
$ gcc -fopenmp -O2 -o exe4_lab5 exe4_lab5.c && OMP_NUM_THREADS=2 ./exe4_la
b5
r = 1, k = 1, thread # = 0
r = 1, k = 2, thread # = 0
r = 1, k = 3, thread # = 0
r = 1, k = 4, thread # = 0
r = 1, k = 5, thread # = 0
r = 1, k = 6, thread # = 0
r = 1, k = 7, thread # = 0
r = 1, k = 8, thread # = 0
r = 1, k = 9, thread # = 0
r = 1, k = 10, thread # = 0
r = 2, k = 1, thread # = 0
r = 2, k = 2, thread # = 0
r = 2, k = 3, thread # = 0
r = 2, k = 4, thread # = 0
r = 2, k = 5, thread # = 0
r = 2, k = 6, thread # = 0
r = 2, k = 7, thread # = 0
r = 2, k = 8, thread # = 0
r = 2, k = 9, thread # = 0
r = 2, k = 10, thread # = 0
r = 3, k = 1, thread # = 0
```

```
r = 1, k = 9, thread # = 0
r = 1, k = 10, thread # = 0
r = 2, k = 1, thread # = 0
r = 2, k = 2, thread # = 0
r = 2, k = 3, thread # = 0
r = 2, k = 4, thread # = 0
r = 2, k = 5, thread # = 0
r = 2, k = 6, thread # = 0
r = 2, k = 7, thread # = 0
r = 2, k = 8, thread # = 0
r = 2, k = 9, thread # = 0
r = 2, k = 10, thread # = 0
r = 3, k = 1, thread # = 0
r = 3, k = 2, thread # = 0
r = 3, k = 3, thread # = 0
r = 3, k = 4, thread # = 0
r = 3, k = 5, thread # = 0
r = 3, k = 6, thread # = 0
r = 3, k = 7, thread # = 0
r = 3, k = 8, thread # = 0
r = 3, k = 9, thread # = 0
r = 3, k = 10, thread # = 0
r = 4, k = 1, thread # = 1
r = 4, k = 2, thread # = 1
r = 4, k = 3, thread # = 1
```

```
r = 5, k = 1, thread # = 1
r = 5, k = 2, thread # = 1
r = 5, k = 3, thread # = 1
r = 5, k = 4, thread # = 1
r = 5, k = 5, thread # = 1
r = 5, k = 6, thread # = 1
r = 5, k = 7, thread # = 1
r = 5, k = 8, thread # = 1
r = 5, k = 9, thread # = 1
r = 5, k = 10, thread # = 1
Time taken: 0.000399 seconds
```

```
vihan@vihanVM in ~ via C v16.1.1-gcc via v5.42.2 via v3.14.5
/ ) gcc -fopenmp -O2 -o exe4_lab5 exe4_lab5.c && OMP_NUM_THREADS=8 ./exe4_la
b5
r = 3, k = 1, thread # = 2
r = 3, k = 2, thread # = 2
r = 3, k = 3, thread # = 2
r = 3, k = 4, thread # = 2
r = 3, k = 5, thread # = 2
r = 3, k = 6, thread # = 2
r = 3, k = 7, thread # = 2
r = 3, k = 8, thread # = 2
r = 3, k = 9, thread # = 2
r = 3, k = 10, thread # = 2
r = 2, k = 1, thread # = 1
r = 2, k = 2, thread # = 1
r = 2, k = 3, thread # = 1
r = 2, k = 4, thread # = 1
r = 2, k = 5, thread # = 1
r = 2, k = 6, thread # = 1
r = 2, k = 7, thread # = 1
r = 2, k = 8, thread # = 1
r = 2, k = 9, thread # = 1
r = 2, k = 10, thread # = 1
r = 4, k = 1, thread # = 3
```

```
r = 4, k = 4, thread # = 3
r = 4, k = 5, thread # = 3
r = 4, k = 6, thread # = 3
r = 4, k = 7, thread # = 3
r = 4, k = 8, thread # = 3
r = 4, k = 9, thread # = 3
r = 4, k = 10, thread # = 3
r = 1, k = 1, thread # = 0
r = 1, k = 2, thread # = 0
r = 1, k = 3, thread # = 0
r = 1, k = 4, thread # = 0
r = 1, k = 5, thread # = 0
r = 1, k = 6, thread # = 0
r = 1, k = 7, thread # = 0
r = 1, k = 8, thread # = 0
r = 1, k = 9, thread # = 0
r = 1, k = 10, thread # = 0
r = 5, k = 1, thread # = 4
r = 5, k = 2, thread # = 4
r = 5, k = 3, thread # = 4
r = 5, k = 4, thread # = 4
r = 5, k = 5, thread # = 4
r = 5, k = 6, thread # = 4
r = 5, k = 7, thread # = 4
r = 5, k = 8, thread # = 4
```

```
r = 1, k = 5, thread # = 0
r = 1, k = 6, thread # = 0
r = 1, k = 7, thread # = 0
r = 1, k = 8, thread # = 0
r = 1, k = 9, thread # = 0
r = 1, k = 10, thread # = 0
r = 5, k = 1, thread # = 4
r = 5, k = 2, thread # = 4
r = 5, k = 3, thread # = 4
r = 5, k = 4, thread # = 4
r = 5, k = 5, thread # = 4
r = 5, k = 6, thread # = 4
r = 5, k = 7, thread # = 4
r = 5, k = 8, thread # = 4
r = 5, k = 9, thread # = 4
r = 5, k = 10, thread # = 4
Time taken: 0.003818 seconds
```

Q. Briefly describe how the threads are generated for this code.

only the *outer* loop (*r*) is parallelized. With 4 threads and 5 values of *r*, OpenMP's default static schedule splits the 5 outer iterations across 4 threads (e.g., thread 0 gets *r*=1,2; thread 1 gets *r*=3; etc.), and each thread runs the entire inner loop (*k*=1..10) serially for its assigned *r* values. You can see this in my output thread 0 owns all of *r*=1 and *r*=2, uninterrupted.

Collapse (2)

```
...on@vihanzVM in ~ via C v16.1.1-gcc via v5.42.2 via v3.14.5 took 14s563ms
) gcc -fopenmp -O2 -o exe4_collapse exe4_collapse.c && OMP_NUM_THREADS=1 .
/exe4_collapse
r = 1, k = 1, thread # = 0
r = 1, k = 2, thread # = 0
r = 1, k = 3, thread # = 0
r = 1, k = 4, thread # = 0
r = 1, k = 5, thread # = 0
r = 1, k = 6, thread # = 0
r = 1, k = 7, thread # = 0
r = 1, k = 8, thread # = 0
r = 1, k = 9, thread # = 0
r = 1, k = 10, thread # = 0
r = 2, k = 1, thread # = 0
r = 2, k = 2, thread # = 0
r = 2, k = 3, thread # = 0
r = 2, k = 4, thread # = 0
r = 2, k = 5, thread # = 0
r = 2, k = 6, thread # = 0
r = 2, k = 7, thread # = 0
r = 2, k = 8, thread # = 0
r = 2, k = 9, thread # = 0
r = 2, k = 10, thread # = 0
r = 3, k = 1, thread # = 0
```

```
r = 2, k = 2, thread # = 0
r = 2, k = 3, thread # = 0
r = 2, k = 4, thread # = 0
r = 2, k = 5, thread # = 0
r = 2, k = 6, thread # = 0
r = 2, k = 7, thread # = 0
r = 2, k = 8, thread # = 0
r = 2, k = 9, thread # = 0
r = 2, k = 10, thread # = 0
r = 3, k = 1, thread # = 0
r = 3, k = 2, thread # = 0
r = 3, k = 3, thread # = 0
r = 3, k = 4, thread # = 0
r = 3, k = 5, thread # = 0
r = 3, k = 6, thread # = 0
r = 3, k = 7, thread # = 0
r = 3, k = 8, thread # = 0
r = 3, k = 9, thread # = 0
r = 3, k = 10, thread # = 0
r = 4, k = 1, thread # = 0
r = 4, k = 2, thread # = 0
r = 4, k = 3, thread # = 0
r = 4, k = 4, thread # = 0
r = 4, k = 5, thread # = 0
r = 4, k = 6, thread # = 0
```

```
r = 4, k = 4, thread # = 0
r = 4, k = 5, thread # = 0
r = 4, k = 6, thread # = 0
r = 4, k = 7, thread # = 0
r = 4, k = 8, thread # = 0
r = 4, k = 9, thread # = 0
r = 4, k = 10, thread # = 0
r = 5, k = 1, thread # = 0
r = 5, k = 2, thread # = 0
r = 5, k = 3, thread # = 0
r = 5, k = 4, thread # = 0
r = 5, k = 5, thread # = 0
r = 5, k = 6, thread # = 0
r = 5, k = 7, thread # = 0
r = 5, k = 8, thread # = 0
r = 5, k = 9, thread # = 0
r = 5, k = 10, thread # = 0
time taken: 0.000368 seconds
```

```
vihan@vihanVM in ~ via C v16.1.1-gcc via v5.42.2 via v3.14.5
$ gcc -fopenmp -O2 -o exe4_collapse exe4_collapse.c && OMP_NUM_THREADS=8 .
/exe4_collapse
r = 1, k = 8, thread # = 1
r = 1, k = 9, thread # = 1
r = 1, k = 10, thread # = 1
r = 2, k = 1, thread # = 1
r = 2, k = 2, thread # = 1
r = 2, k = 3, thread # = 1
r = 2, k = 4, thread # = 1
r = 3, k = 7, thread # = 4
r = 3, k = 8, thread # = 4
r = 3, k = 9, thread # = 4
r = 3, k = 10, thread # = 4
r = 4, k = 1, thread # = 4
r = 4, k = 2, thread # = 4
r = 4, k = 9, thread # = 6
r = 4, k = 10, thread # = 6
r = 5, k = 1, thread # = 6
r = 5, k = 2, thread # = 6
r = 5, k = 3, thread # = 6
r = 5, k = 4, thread # = 6
r = 5, k = 5, thread # = 7
r = 4, k = 3, thread # = 5
```

```

r = 4, k = 1, thread # = 4
r = 4, k = 2, thread # = 4
r = 4, k = 9, thread # = 6
r = 4, k = 10, thread # = 6
r = 5, k = 1, thread # = 6
r = 5, k = 2, thread # = 6
r = 5, k = 3, thread # = 6
r = 5, k = 4, thread # = 6
r = 5, k = 5, thread # = 7
r = 4, k = 3, thread # = 5
r = 4, k = 4, thread # = 5
r = 4, k = 5, thread # = 5
r = 4, k = 6, thread # = 5
r = 4, k = 7, thread # = 5
r = 4, k = 8, thread # = 5
r = 5, k = 6, thread # = 7
r = 5, k = 7, thread # = 7
r = 5, k = 8, thread # = 7
r = 5, k = 9, thread # = 7
r = 5, k = 10, thread # = 7
r = 2, k = 5, thread # = 2
r = 2, k = 6, thread # = 2
r = 2, k = 7, thread # = 2
r = 2, k = 8, thread # = 2
r = 2, k = 9, thread # = 2

```

```
r = 5, k = 9, thread # = 7
r = 5, k = 10, thread # = 7
r = 2, k = 5, thread # = 2
r = 2, k = 6, thread # = 2
r = 2, k = 7, thread # = 2
r = 2, k = 8, thread # = 2
r = 2, k = 9, thread # = 2
r = 2, k = 10, thread # = 2
r = 1, k = 1, thread # = 0
r = 1, k = 2, thread # = 0
r = 1, k = 3, thread # = 0
r = 1, k = 4, thread # = 0
r = 1, k = 5, thread # = 0
r = 1, k = 6, thread # = 0
r = 1, k = 7, thread # = 0
r = 3, k = 1, thread # = 3
r = 3, k = 2, thread # = 3
r = 3, k = 3, thread # = 3
r = 3, k = 4, thread # = 3
r = 3, k = 5, thread # = 3
r = 3, k = 6, thread # = 3
Time taken: 0.002145 seconds
```

Q . Replace the code below with `#pragma omp parallel for collapse(2)` how are the threads generated.

OpenMP flattens the two nested loops into one logical loop of $5 \times 10 = 50$ iterations, then divides *that* combined iteration space across the 4 threads. My output shows this clearly thread 0 no longer owns whole rows; it gets a contiguous chunk of about 12-13 of the 50 total (r,k) pairs, which can cut across row boundaries (e.g., thread 1 picks up partway through $r=2$). collapse gives finer-grained, more even load balancing when the outer loop trip count is small relative to the thread count.